Learn Physics with Functional Programming

—

Scott N. Walck

Graham Strickland

November 3, 2026

4 Describing Motion

4.2 For $f(x) = x^3$, we have $f'(x) = 3x^2$, so that the relative error is defined by

$$\begin{aligned}
\text{err}(x,a) &= \left| \frac{\frac{f(x+a/2)-f(x-a/2)}{a} - f'(x)}{f'(x)} \right| \\
&= \left| \frac{\frac{(x+a/2)^3-(x-a/2)^3}{a} - 3x^2}{3x^2} \right| \\
&= \left| \frac{\begin{array}{l}[x^3 + (3x^2a)/2 + (3xa^2)/4 + a^3/8] \\ \quad - [x^3 - (3x^2a)/2 + (3xa^2)/4 - a^3/8]\end{array}}{\frac{a}{3x^2}} - 3x^2 \right| \\
&= \left| \frac{\frac{3x^2a+a^3/4-3x^2a}{a}}{3x^2} \right| \\
&= \left| \frac{\frac{a^2}{4}}{3x^2} \right| \\
&= \left| \frac{a^2}{12x^2} \right| \\
&= \frac{a^2}{12x^2},
\end{aligned}$$

since $a^2 \geq 0$ and $x^2 \geq 0$.

Thus we have an error of 1 percent if

$$\begin{aligned}
&\mathrm{err}(x, a) = 0.01 \\
\Leftrightarrow\ &\frac{a^2}{12x^2} = 0.01 \\
\Leftrightarrow\ &a^2 = 0.12x^2 \\
\Leftrightarrow\ &a = |x|\sqrt{0.12}
\end{aligned}$$

Then, for $x = 4$, we have

$$\begin{aligned}
a &= 4\sqrt{0.12} \\
&\approx 1.3856406460551018
\end{aligned}$$

and for $x = 0.1$, we have

$$\begin{aligned}
a &= 0.1\sqrt{0.12} \\
&\approx 3.4641016151377546 \times 10^{-2}.
\end{aligned}$$

4.3 Suppose we have a function f and independent variable, say x, such that `derivative 0.01 f x` produces at least a 10 percent error, $\mathrm{err}(x, \epsilon)$, compared to the exact derivative, $f'(x)$. Then, we have

$$\begin{aligned}
\mathrm{err}(x, \epsilon) &= \mathrm{err}(x, 0.01) \\
&= \left| \frac{\frac{f(x+\epsilon/2)-f(x-\epsilon/2)}{\epsilon} - f'(x)}{f'(x)} \right| \\
&= \left| \frac{\frac{f(x+0.01/2)-f(x-0.01/2)}{0.01} - f'(x)}{f'(x)} \right| \\
&= \left| \frac{\frac{f(x+0.005)-f(x-0.005)}{0.01} - f'(x)}{f'(x)} \right| \\
&\geq 0.1.
\end{aligned}$$

If we substitute $\epsilon = 0.01$, $x = \pi \approx 3.141592653589793$, and $f(x) = \cos x$ into the above, we have $f'(x) = -\sin x$, so that

$$\begin{aligned}
\mathrm{err}(x, \epsilon) &= \mathrm{err}(\pi, 0.01) \\
&= \left| \frac{\frac{\cos(\pi+0.005)-\cos(\pi-0.005)}{0.01} + \sin\pi}{-\sin\pi} \right| \\
&\approx 1.0 \\
&\geq 0.1.
\end{aligned}$$

4.4 We cannot apply our error function in 4.3 to `derivative a cos`, since it results in division by 0, but at values close to `t = 0`, we see an initial increase in the error as we move from small values of `a` which stop increasing as we increase `a` by multiples of 10 past `a = 10`.

For the following definition of the error function

```
err :: (R -> R) -> (R -> R) -> R -> R -> R
err f df t a = abs ((derivative a f t - df t) / df t)
```

we have the following output from `GHCi`:

```
ghci> err (\x -> cos x) (\x -> -sin x) 0.1 0.01
4.166661374023121e-6
ghci> err (\x -> cos x) (\x -> -sin x) 0.1 0.1
4.166145864253412e-4
ghci> err (\x -> cos x) (\x -> -sin x) 0.1 1
4.114892279159413e-2
ghci> err (\x -> cos x) (\x -> -sin x) 0.1 10
1.191784854932627
ghci> err (\x -> cos x) (\x -> -sin x) 0.1 100
1.0052474970740786
ghci> err (\x -> cos x) (\x -> -sin x) 0.1 1000
1.000935543610645
ghci> err (\x -> cos x) (\x -> -sin x) 0.1 10000
1.000197593287754
ghci> err (\x -> cos x) (\x -> -sin x) 0.1 100000
1.0000199968037815
ghci> err (\x -> cos x) (\x -> -sin x) 0.1 1000000
0.999999644337597
ghci> err (\x -> cos x) (\x -> -sin x) 0.1 10000000
1.000000195308493
```

5 Working with Lists

5.4 We have the function `range` defined in `lpfp/src/Ch05/Range.hs`.

`range` returns a list containing all the integers between the argument (inclusive) and 0 in increasing order, i.e, `range`$(2) = 0, 1, 2$, `range`$(-4) = -4, -3, \ldots, 0$, and `range`$(0) = 0$.

We demonstrate as follows:

```
ghci> range (-4)
[-4,-3,-2,-1,0]
ghci> range 2
[0,1,2]
ghci> range 0
[0]
```

5.8 We find the first five elements of the infinite list $[9, 1, \ldots]$ as follows:

```
ghci> take 5 [9,1..]
[9,1,-7,-15,-23]
```

Thus we see that the first five elements are given by

$$[9, 1, \ldots] = [9, 1, -7, -15, -23, \ldots].$$

5.9 We have the function `cycle'` with definition in `lpfp/src/Ch05/Cycle.hs`.

`cycle'` repeats an argument which implements `HasCallStack` an infinite number of times.

We demonstrate as follows:

```
ghci> take 10 (cycle' [4,7,8])
[4,7,8,4,7,8,4,7,8,4]
ghci> take 10 (cycle' [1])
[1,1,1,1,1,1,1,1,1,1]
```

5.10 (a) `["hello",42]` is not a valid expression, since it attempts to construct a list from elements of two different types, `String` and —Int—.

(b) `['h',"ello"]` is not a valid expression, since it attempts to construct a list from elements of two different types, `Char` and —String—.

(c) `['a','b','c']` is a valid expression.

(d) `length ['w','h','o']` is a valid expression.

(e) `length "hello"` is a valid expression.

(f) `reverse` is a valid expression, even though GHCI cannot print it.

5.11 It seems as if an arithmetic sequence will end at the last integer in the sequence before the last element in the constructor if the sequence is an integer sequence.

If it is a floating point sequence, i.e., one of the elements in the constructor was of floating point type, then the last number in the sequence will be the number in the sequence occurring after the last element in the constructor if that last element is further than the midpoint between two elements in the sequence, otherwise it will be the number occurring before.

We demonstrate as follows:

```
ghci> [0,3..7.5]
[0.0,3.0,6.0,9.0]
ghci> [0,3..7.49]
[0.0,3.0,6.0]
ghci> [0,3..7.499999999]
[0.0,3.0,6.0]
ghci> [0,3..7]
[0,3,6]
ghci> [0,3..8]
[0,3,6]
ghci> [0,3..9]
[0,3,6,9]
```

5.12 We define the expression in `lpfp/src/Ch05/GeometricSeries.hs` used to calculate

$$\sum_{n=1}^{100} \frac{1}{n^2}.$$

Evaluating this in `GHCi` results in the following:

```
ghci> series
5.187377517639621
```

5.14 We have the definitions of these functions in the file `lpfp/src/Ch05/Exponential.hs`.

In order to calculate how big n needs to be to get within 1 percent of the correct value for $x = 1$, we have the following:

```
ghci> calcMinExpErr 1 1 0.01
134
```

To calculate how big n needs to be to get within 1 percent of the correct value for $x = 10$, we have:

```
ghci> calcMinExpErr 1 10 0.01
```

This does not return a value within 10 minutes.

5.15 For this question, we update the code in `lpfp/src/Ch05/Exponential.hs` to allow us to pass in a different approximation function, in this case the function `expSeries` and calculate the result. Thus we have

```
type R = Double

expList :: R -> [R]
expList x = [(1.0 + x / n) ** n | n <- [1 ..]]

expSeries :: R -> [R]
expSeries x = [(x ** n) / product [1 .. n] | n <- [1 ..]]

expErr :: R -> R -> R
expErr x approx = abs (exp x - approx)

calcMinExpErr :: Int -> R -> R -> (R -> [R]) -> Int
calcMinExpErr n x eps approx =
  if expErr x (approx x !! n) < eps
    then n
    else calcMinExpErr (n + 1) x eps approx
s approx
```

However, this does not return a result even for $x = 1$ within any reasonable time frame, so these functions must be optimized to produce a result more quickly.

6 Higher-Order Functions

6.1 We define the functions in `lpfp/src/Ch06/RockTrajectory.hs`.

The first corresponds to the equation

$$y = v_0 t - \frac{1}{2} g t^2$$

and the second to

$$v = v_0 - gt.$$

6.2 We have the following

```
ghci> :t take 4
take 4 :: [a] -> [a]
```

6.3 We have the following

```
ghci> :t map
map :: (a -> b) -> [a] -> [b]
ghci> :t not
not :: Bool -> Bool
ghci> :t map not
map not :: [Bool] -> [Bool]
```

where `not` substitutes the type `Bool` for `a` and `b` in the definition of `map`, so that `map not` has the type `[Bool] -> [Bool]`.

6.5 We have the required definition in the file `lpfp/src/Ch06/IntStrBool.hs`.

The function `intStringBool` takes an `Int` n and returns a function that checks if any ASCII character in a string is equivalent to the number n and returns `True` if it is and `False` if it does not find a match. We demonstrate as follows

```
ghci> intStringBool 48 "Hello1"
False
ghci> intStringBool 49 "Hello1"
True
```

6.7 The function `replicate` has the following definition

```
ghci> :t replicate
replicate :: Int -> a -> [a]
```

which indicates that it takes an input of type `Int` and an input of type `a` and produces a list of type `[a]`. The first three examples use types that are concatenated into lists. The fourth example uses the value `'x'` of type `Char`, which when concatenated together 3 times produces a list of `Char` which is simplified to the equivalent `String`, since a `String` is a list of `Char`.

6.11 We have the following test using GHCi

```
ghci> take 10 (iterate (\t -> t + 5) 0)
[0,5,10,15,20,25,30,35,40,45]
```

6.15 Table 6.1 explains the two ways of thinking about the higher-order function `drop` and Table 6.2 provides the same for `replicate`:

Way of thinking	**Input to** `drop`	**Output from** `drop`
One-input thinking	`Int`	`[a] -> [a]`
Two-input thinking	`Int` and then `[a]`	`[a]`

Table 6.1: Two Ways of Thinking About the Higher-Order Function `drop`

Way of thinking	**Input to** `replicate`	**Output from** `replicate`
One-input thinking	`Int`	`a -> [a]`
Two-input thinking	`Int` and then `a`	`[a]`

Table 6.2: Two Ways of Thinking About the Higher-Order Function `replicate`

6.16 We have the the definition of the functions for the trapezoidal rule in the file `lpfp/src/Ch06/TrapezoidalRule.hs`, which we test using the helper function `calcTrapIntegrateErr`, so that we have

```
ghci> calcTrapIntegrateErr 1 (\x -> x ^ 3) 0 1 0.001
4
ghci> calcTrapIntegrateErr 1 (\x -> x ^ 3) 0 1e-6 0.001
2
ghci> calcTrapIntegrateErr 1 (\x -> exp (-x^2)) 0 1 0.001
3
```

7 Graphing Functions

7.1 Using the definition in the file `lpfp/src/Ch07/SinPlot.hs`, we have

```
ghci> :l
Ok, no modules loaded.
ghci> :m
ghci> :l Ch07.SinPlot
[1 of 1] Compiling Ch07.SinPlot
( lpfp/src/Ch07/SinPlot.hs, interpreted )
Ok, one module loaded.
ghci> sinPlot
```

to produce the plot in Figure 7.1.

7.2 Using the definition in the file `lpfp/src/Ch07/PlotYRock30.hs`, we produce the plot in Figure 7.2.

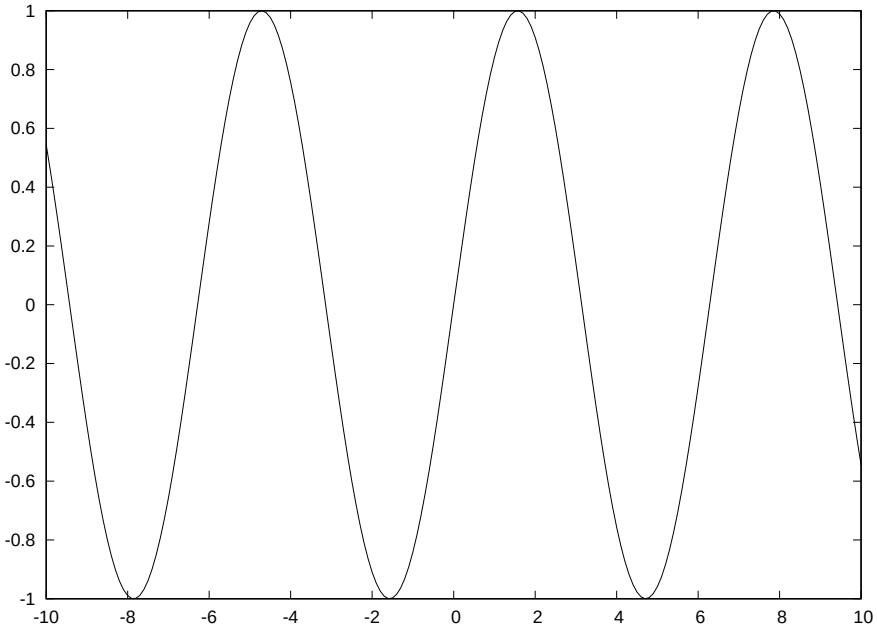


Figure 7.1: Plot of $sin(x)$ from $x = -10$ to $x = 10$

7.3 Using the definition in the file `lpfp/src/Ch07/PlotYRock30.hs`, we produce the plot in Figure 7.3.

8 Type Classes

8.1 Yes, for example, the type `Float` belongs to both `Floating` and `Fractional`.

8.2 (a) Two functions f and g are equal if and only if they have the same domain and range, and for every element of the domain, they map to the same element of the range, i.e., if $f : X \to Y$ and $g : X \to Y$, then

$$f = g$$
$$\Leftrightarrow f(x) = g(x) \forall x \in X.$$

(b) The computer cannot necessarily compute each element of the domain in a reasonable time to ensure that both functions map each element of the domain to the same element in the range.

(c)
```
f :: Bool a => a -> a
```

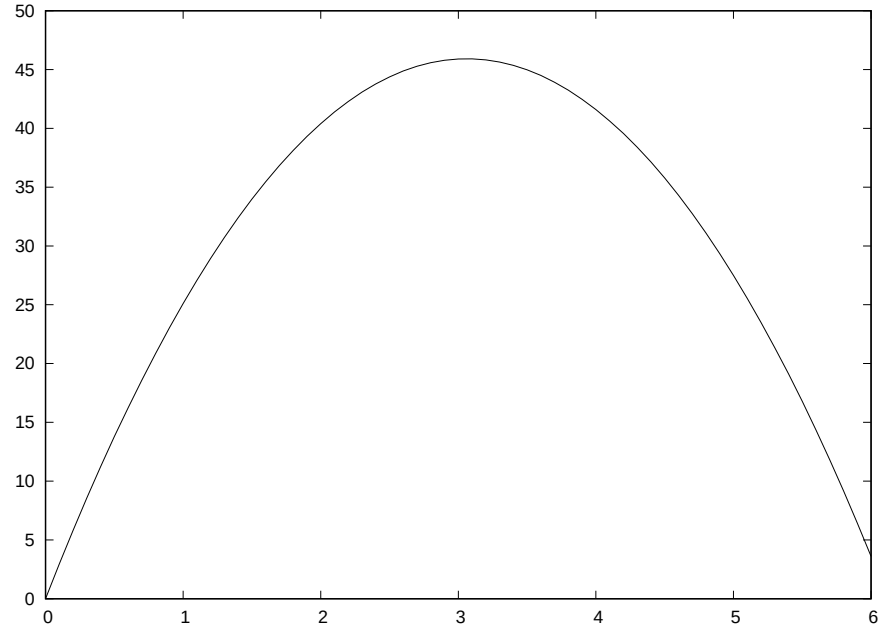


Figure 7.2: Plot of `yRock30` function from $t = 0$ to $t = 6s$

8.3 No, the function `(/2)` computes division by 2, while the function `(2/)` computes division by 2 of another argument, e.g., $1/2 \neq 2/1$.

8.4 Depending upon which type of base we are squaring, we could use the sections `(^2)`, `(^^)`, or `(**)`.

8.5 (a) We have the following from GHCi:

```
ghci> :i Integer
type Integer :: *
data Integer
  = GHC.Num.Integer.IS ghc-prim-0.9.1:GHC.Prim.Int#
  | GHC.Num.Integer.IP ghc-prim-0.9.1:GHC.Prim.ByteArray#
  | GHC.Num.Integer.IN ghc-prim-0.9.1:GHC.Prim.ByteArray#
        -- Defined in ‘GHC.Num.Integer’
instance Integral Integer -- Defined in ‘GHC.Real’
instance Num Integer -- Defined in ‘GHC.Num’
instance Real Integer -- Defined in ‘GHC.Real’
instance Enum Integer -- Defined in ‘GHC.Enum’
instance Eq Integer -- Defined in ‘GHC.Num.Integer’
instance Ord Integer -- Defined in ‘GHC.Num.Integer’
instance Read Integer -- Defined in ‘GHC.Read’
instance Show Integer -- Defined in ‘GHC.Show’
```

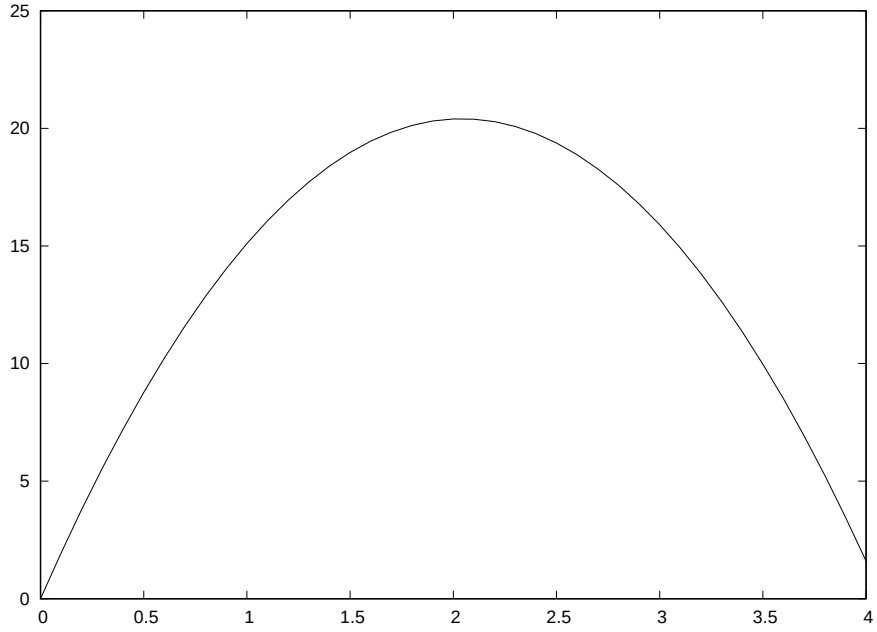


Figure 7.3: Plot of `yRock20` function from $t = 0$ to $t = 4s$

Thus we can see that `Integer` also belongs to `Real`, indicating that for an integer $x \in \mathbb{Z}$, we have $x \in \mathbb{R}$, since $\mathbb{Z} \subseteq \mathbb{R}$. We also see that `Integer` belongs to `Enum`, which represents the set of enumerable (countable) numbers. Finally, `Integer` also belongs to `Read`, which is the input type corresponding to `Show`, indicating that elements of `Read` can be serialized.

(b) Executing the GHCi command `:info` yields:

```
ghci> :i Enum
type Enum :: * -> Constraint
class Enum a where
  succ :: a -> a
  pred :: a -> a
  toEnum :: Int -> a
  fromEnum :: a -> Int
  enumFrom :: a -> [a]
  enumFromThen :: a -> a -> [a]
  enumFromTo :: a -> a -> [a]
  enumFromThenTo :: a -> a -> a -> [a]
  {-# MINIMAL toEnum, fromEnum #-}
        -- Defined in ‘GHC.Enum’
instance Enum Double -- Defined in ‘GHC.Float’
instance Enum Float -- Defined in ‘GHC.Float’
instance Enum () -- Defined in ‘GHC.Enum’
```

```
instance Enum Bool -- Defined in ‘GHC.Enum’
instance Enum Char -- Defined in ‘GHC.Enum’
instance Enum Int -- Defined in ‘GHC.Enum’
instance Enum Integer -- Defined in ‘GHC.Enum’
instance Enum Ordering -- Defined in ‘GHC.Enum’
instance Enum a => Enum (Solo a) -- Defined in ‘GHC.Enum’
instance Enum Word -- Defined in ‘GHC.Enum’
```

Thus we see that `Double`, `Float`, `()`, `Bool`, `Char`, `Int`, `Integer`, `Ordering`, `Solo`, and `Word` are instances of `Enum`.

8.6 (a) `42 :: Num a => a`

(b) `42.0 :: Fractional a => a`

(c) `42.5 :: Fractional a => a`

(d) `pi :: Floating a => a`

(e) `[3,1,4] :: Num a => [a]`

(f) `[3,3.5,4] :: Fractional a => [a]`

(g) `[3,3.1,pi] :: Floating a => [a]`

(h) `(==) :: Eq a => a -> a -> Bool`

(i) `(/=) :: Eq a => a -> a -> Bool`

(j) `(<) :: Ord a => a -> a -> Bool`

(k) `(<=) :: Ord a => a -> a -> Bool`

(l) `(+) :: Num a => a -> a -> a`

(m) `(-) :: Num a => a -> a -> a`

(n) `(*) :: Num a => a -> a -> a`

(o) `(/) :: Fractional a => a -> a -> a`

(p) `(^) :: (Num a, Integral b) => a -> b -> a`

(q) `(**) :: Floating a => a -> a -> a`

(r) `8/4 :: Fractional a => a`

(s) `sqrt :: Floating a => a -> a`

(t) `cos :: Floating a => a -> a`

(u) `show :: Show a => a -> String`

(v) `(2/) :: Fractional a => a -> a`

8.7 Because the operator `/` has type `(/) :: Fractional a => a -> a -> a`.

8.8 Using the helper functions in `lpfp/src/Ch08/Quotients.hs`, we see that

```
ghci> quot (-4) 3 == div (-4) 3
False
```

and

```
ghci> quot (-4) 3
-1
ghci> div (-4) 3
-2
```

Thus we can conclude that for integer division with negative numerators, `quot` rounds down to the next integer in the positive direction, while `div` rounds down in the negative direction.

Likewise, we see that

```
ghci> rem (-4) 3 == mod (-4) 3
False
```

and

```
ghci> rem (-4) 3
-1
ghci> mod (-4) 3
2
```

Thus we conclude that when calculating remainders with negative numerators, `rem` returns the negative of the remainder when the modulus of the numerator is divided by the denominator, while `mod` calculates a where $a \equiv m (\mathrm{mod}\, n)$, for m the argument on the left and n that on the right.

8.9 Table 8.1 shows which types can be used for the base `x` and the exponent `y` in the expresion `x ^ y` while Table 8.2 does the same for the expression `x ** y`.

	`y :: Int`	`y :: Integer`	`y :: Float`	`y :: Double`
`x :: Int`	`^`	`^`		
`x :: Integer`	`^`	`^`		
`x :: Float`	`^`	`^`		
`x :: Double`	`^`	`^`		

Table 8.1: Possible Types for `x` and `y` with the Single-Caret Exponentiation Operator

	`y :: Int`	`y :: Integer`	`y :: Float`	`y :: Double`
`x :: Int`				
`x :: Integer`				
`x :: Float`			`**`	
`x :: Double`				`**`

Table 8.2: Possible Types for `x` and `y` with the Double-Asterisk Exponentiation Operator

We can see from all three tables that there is no applicable exponentiation operator if the base has type `Float` and the exponent type `Double`.

9 Tuples and Type Constructors

9.2 `curry` takes as input a function which takes a tuple `(a, b)` as input and outputs a value of type `c` and returns a function which takes in two variables `a` and `b` and outputs a value `c`. `uncurry` does the opposite.

9.5 The remaining elements of the longer list are discarded, e.g.,

```
ghci> zip [1] [1,2]
[(1,1)]
```

9.7 No, for instance, we have the following error when trying to convert a list of empty tuples (the list containing the unit type):

```
ghci> (zip' . unzip) [()]

<interactive>:10:17: error:
    • Couldn't match expected type ‘(a, b)’ with actual type ‘()’
    • In the expression: ()
      In the first argument of ‘zip' . unzip’, namely ‘[()]’
      In the expression: (zip' . unzip) [()]
    • Relevant bindings include
        it :: [(a, b)] (bound at <interactive>:10:1)
```

Yes, since this does return exactly the expression it was passed in, e.g.,

```
ghci> (unzip . zip') ([], [])
([],[])
ghci> (unzip . zip') ([1,2,3], ["Albert","Isaac","James"])
([1,2,3],["Albert","Isaac","James"])
```

9.8 We have the following:

```
ghci> lookup "Albert Einstein" grades
Just 89
ghci> lookup "James Clerk Maxwell" grades
Nothing
```

9.14 Using the definitions from `lpfp/src/Ch09/RockTrajectory.hs`, we produce the plot in Figure 9.1.

10 Describing Motion in Three Dimensions

10.1 (a)
```
v0 :: Vec
v0 = 20 *^ iHat
```

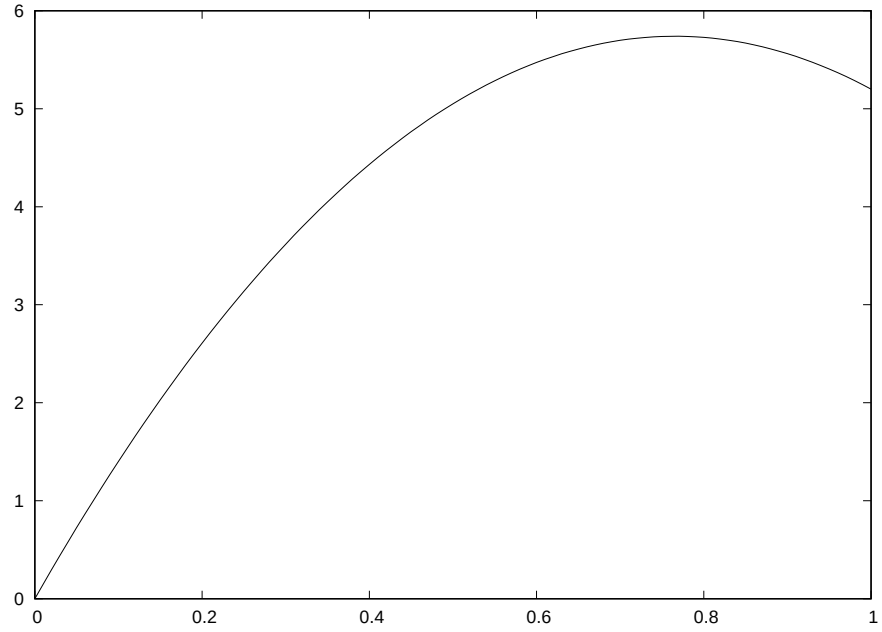


Figure 9.1: Plot of t and y values for `rockTrajectory`

```
v1 :: Vec
v1 = 20 *^ iHat ^-^ 9.8 *^ kHat
```

(b)
```
v :: R -> Vec
v t = 20 *^ iHat ^-^ 9.8 *^ t *^ kHat

r :: R -> Vec
r t = 30 *^ jHat ^+^ 20 *^ t *^ iHat ^-^ 4.9 *^ t ** 2 *^ kHat
```

(d)
```
x :: R -> R
x t = iHat <.> r t
```

10.2 Using the definition from `lpfp/src/Ch10/VecIntegral.hs`, we have

```
ghci> vecIntegral 0.01 v1 0 10
vec 6.666665000000005 0.7499996250000007 1.9999983333336274e-2
```

which gives an approximation to the integral from 0 to 10 with respect to time for the vector function

$$\mathbf{v}_1(t) = 2t^2\hat{\mathbf{i}} + 3t^3\hat{\mathbf{j}} + t^4\hat{\mathbf{k}}.$$

10.3 Using the definition from `lpfp/src/Ch10/MaxHeight.hs`, we have

```
ghci> maxHeight (vec 1.0 1.0 1.0) (5.0 *^ kHat)
2.2737499999999997
```

10.4 Using the definition from `lpfp/src/Ch10/Speed.hs`, we have

```
ghci> speedCA (Vec 1.0 1.0 1.0) (9.81 *^ negateV kHat) 1.0
8.922785439536243
```

10.5 Using the definition from `lpfp/src/Ch10/ProjectileVel.hs`, we have

```
ghci> projectileVel (Vec 1.0 1.0 1.0) 1.0
vec 1.0 1.0 (-8.81)
ghci> projectileVel (Vec 1.0 1.0 1.0) 2.0
vec 1.0 1.0 (-18.62)
```

10.6 The `Vec2D` class is contained in `lpfp/src/Ch10/Vec2D.hs`. We demonstrate the use of `magAngleFromVec2D` and `vec2DFromMagAngle` using

```
ghci> magAngleFromVec2D jHat
(1.0,1.5707963267948966)
ghci> magAngleFromVec2D iHat
(1.0,0.0)
ghci> magAngleFromVec2D (negateV iHat)
(1.0,-3.141592653589793)
ghci> magAngleFromVec2D (negateV jHat)
(1.0,-1.5707963267948966)
ghci> vec2DFromMagAngle (1.0,-1.5707963267948966)
vec 6.123233995736766e-17 (-1.0)
ghci> vec2DFromMagAngle (1.0,0.0)
vec 1.0 0.0
ghci> vec2DFromMagAngle (1.0,0.0)
```

10.9 Using the definition in the file `lpfp/src/Ch10/BallSpeed.hs`, we have plot the z component of the ball's trajectory in Figure 10.1 and the rate of change of its speed in Figure 10.2.

From this plot, we can tell that the rate of change of its speed is equal when it reachs time $t = 4$s. At this point its acceleration is, by definition $a = 0$, while its velocity is given by $v \approx 61.56745393019432$.

10.11 We have the following output using the functions defined in `lpfp/src/Ch10/NonuniformCircularMotion.hs`

```
ghci> radialComponent 2 2
vec (-69.11249319950134) 20.10935225661961 0.0
ghci> radialComponent 2 2
ghci> vNCM 0.01 (2, thetaFunc) 2
11.999999999999744
ghci> magRadialComponent 2 2
71.9786271363377
ghci> squareSpeedDivRad 2 2
71.99999999999693
```

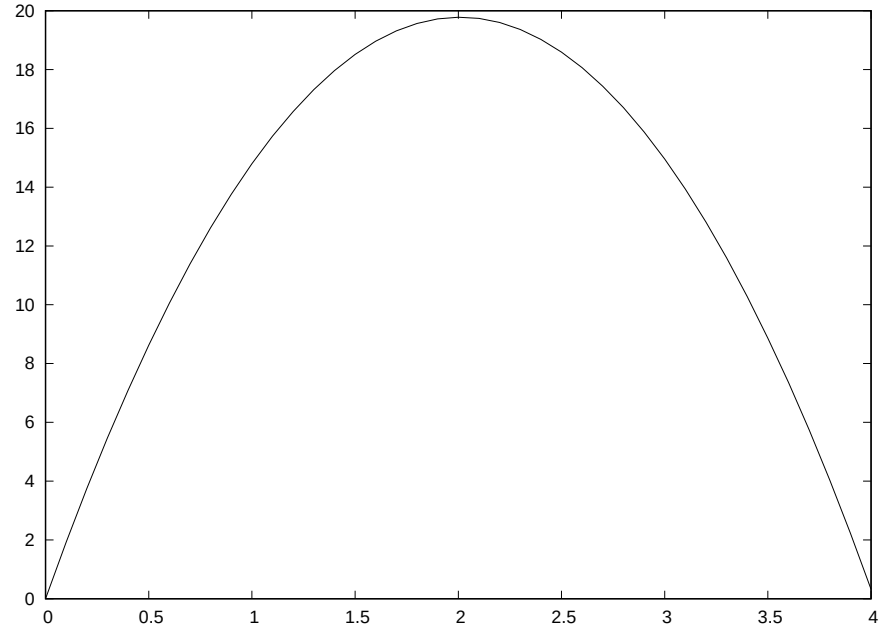


Figure 10.1: Plot of the z component of the ball's trajectory as a function of time

Thus we can conclude that the radial component of acceleration at $t = 2$s is given by the vector

$$\begin{aligned}\mathbf{a}_r\left(R, \theta(t), t\right) &= \mathbf{a}_r\left(2, \theta(2), 2\right) \\ &\approx (-69.11249319950134, 20.10935225661961, 0.0),\end{aligned}$$

the speed of the particle at time $t = 2$s is given by

$$\begin{aligned}v_{\mathrm{NCM}}\left(R, \theta(t), t\right) &= v_{\mathrm{NCM}}\left(2, \theta(2), 2\right) \\ &\approx 11.999999999999744,\end{aligned}$$

and the magnitude of the radial component at that time by

$$\begin{aligned}\left|\mathbf{a}_r\left(R, \theta(t), t\right)\right| &= \left|\mathbf{a}_r\left(2, \theta(2), 2\right)\right| \\ &\approx 71.99999999999693.\end{aligned}$$

11 Creating Graphs

11.1 In Figure 11.1, we have a plot of the function $y = x^2$ for $x \in [-3, 3]$.

11.2 In Figure 11.2, we have a plot of the functions cos and sin for $x \in [0, 10]$.

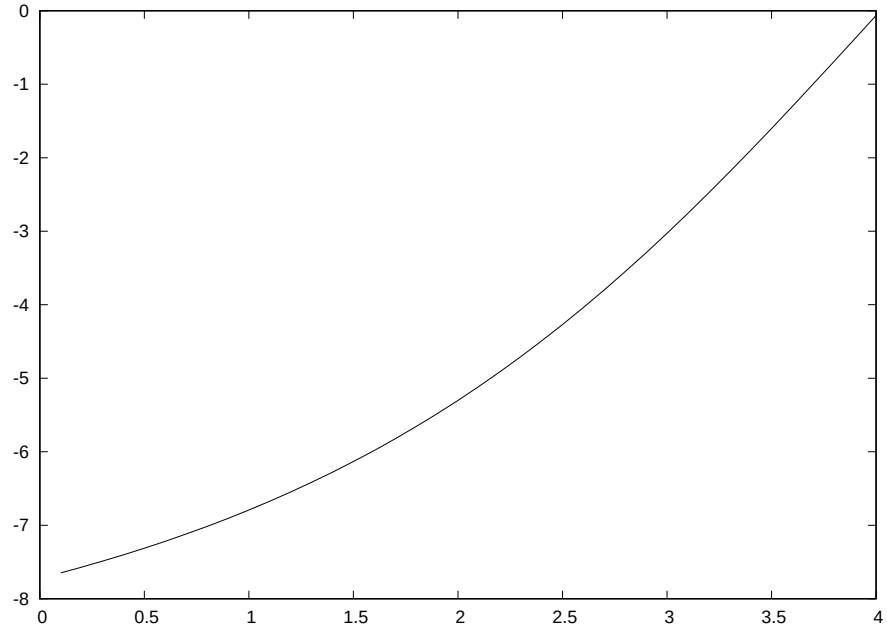


Figure 10.2: Plot of rate of change of the speed of the ball as a function of time

11.3 In Figure 11.3, we have a plot of $x(t) = 30t - 4.9t^2$ for $t \in [0, 6]$ in units of seconds and x in units of meters.

11.4 In Figure 11.4, we have a plot of the first four terms of the Taylor approximation

$$f(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!}$$

for $x \in [-4, 4]$ along with the actual function $\sin x$ over the same domain.

14 Newton's Second Law and Differential Equations

14.6 We have the following output from `GHCi`:

```
ghci> velocityFv 0.5 70 0 [const 100,\v -> fAirT 2 1.225 0.6 (1.5,
↪ v)] 1.5
2.1295044266183036
```

14.7 We have the following output from `GHCi`:

```
ghci> velocityFtv 6 20 (0,0) [\(t,_) -> pedalCoast t,\(_,v) -> fAir
↪  2 1.225 0.5 v] 18
```

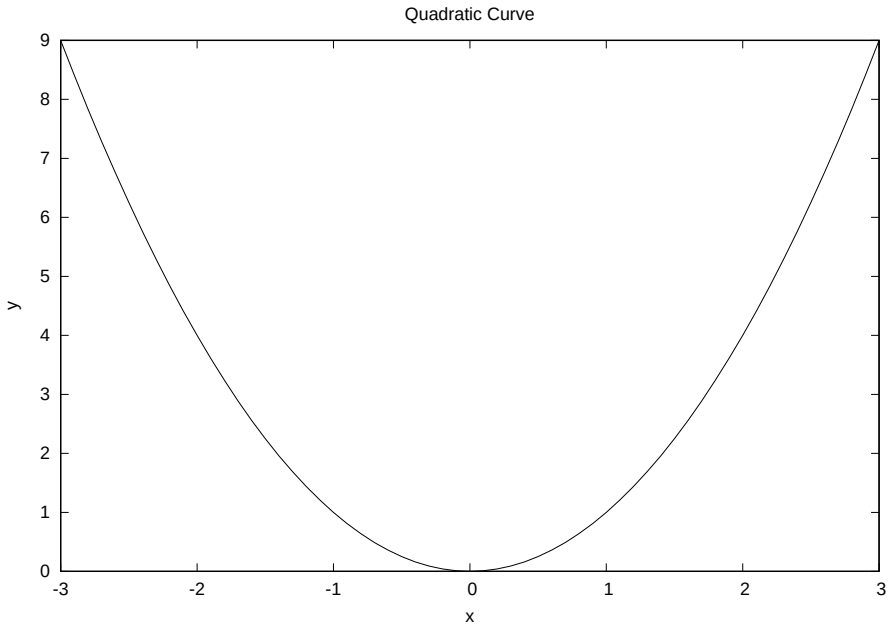


Figure 11.1: Plot of $y = x^2$

```
0.8752328847656248
```

14.8 In Figure 14.1, we have a plot of the velocity of the bike with a step value of 1s, so that the function now appears smooth.

14.10 In Figure 14.2, we have a plot of the position of the bike while pedalling and then coasting, with air resistance.

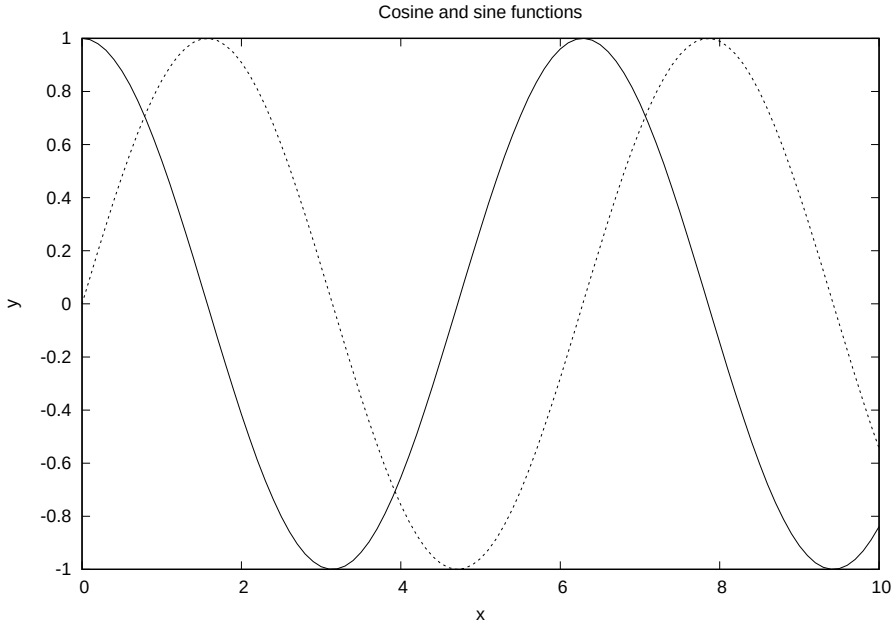


Figure 11.2: Plot of *cos* and *sin*

14.11 Applying (14.16) repeatedly to the given figures, we have

$$\begin{aligned}
v(0.1\,\mathrm{s}) &= v(0\,\mathrm{s}) + \frac{F_{net}^{tv}\big(0\,\mathrm{s}, v(0\,s)\big)(0.1\,\mathrm{s})}{1\mathrm{kg}} \\
&= 2.0000\,\mathrm{m/s} + \big(F_1(0) + F_2(2.0000)\mathrm{m/s}^2\big)(0.1\,\mathrm{s}) \\
&= 2.0000\,\mathrm{m/s} + \big(4\cos 0 - 3(2.0000)\mathrm{m/s}^2\big)(0.1\,\mathrm{s}) \\
&= 1.8000\,\mathrm{m/s} \\
v(0.2\,\mathrm{s}) &= v(0.1\,\mathrm{s}) + \frac{F_{net}^{tv}\big(0.1\,\mathrm{s}, v(0.1\,s)\big)(0.1\,\mathrm{s})}{1\mathrm{kg}} \\
&= 1.8000\,\mathrm{m/s} + \big(F_1(0.1) + F_2(1.8000)\mathrm{m/s}^2\big)(0.1\,\mathrm{s}) \\
&= 1.8000\,\mathrm{m/s} + \big(4\cos 0.1 - 3(1.8000)\mathrm{m/s}^2\big)(0.1\,\mathrm{s}) \\
&= 1.6580\,\mathrm{m/s} \\
v(0.3\,\mathrm{s}) &= v(0.2\,\mathrm{s}) + \frac{F_{net}^{tv}\big(0.2\,\mathrm{s}, v(0.2\,s)\big)(0.1\,\mathrm{s})}{1\mathrm{kg}} \\
&= 1.6580\,\mathrm{m/s} + \big(F_1(0.2) + F_2(1.6580)\mathrm{m/s}^2\big)(0.1\,\mathrm{s}) \\
&= 1.6580\,\mathrm{m/s} + \big(4\cos 0.2 - 3(1.6580)\mathrm{m/s}^2\big)(0.1\,\mathrm{s}) \\
&= 1.5526\,\mathrm{m/s}.
\end{aligned}$$

The figures are tabulated in Table 14.1.

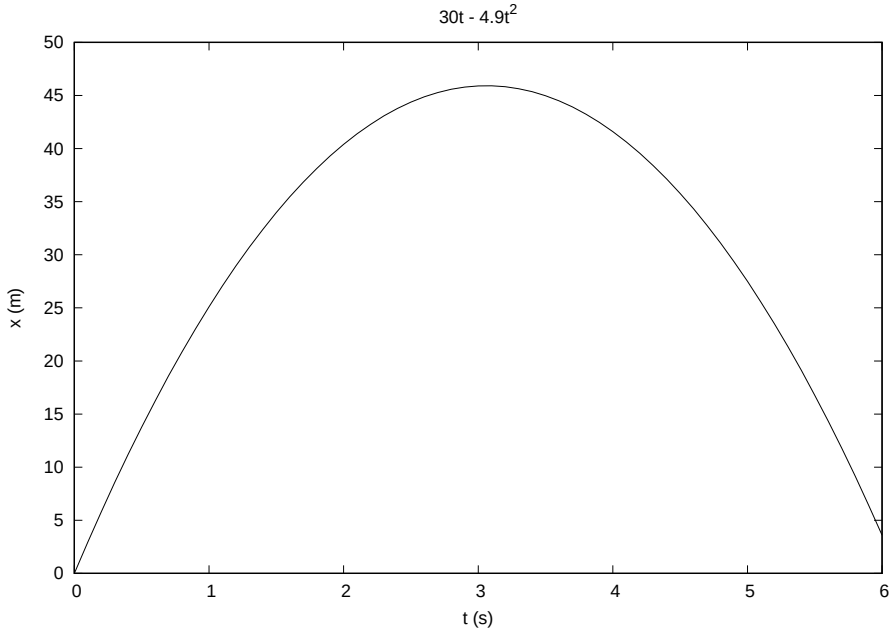


Figure 11.3: Plot of $x(t) = 30t - 4.9t^2$

t (s)	v(t) (m/s)
0	2.0000
1	1.8000
2	1.6580
3	1.5526

Table 14.1: Values using Euler method

14.12 We have the following results from `GHCi`:

```
ghci> updateExample (0,2)
(0.1,1.8)
ghci> updateExample (0.1,1.8)
(0.2,1.6580016661112102)
ghci> updateExample (0.2,1.6580)
(0.30000000000000004,1.5526266311364965)
```

These results agree with those contained in Table 14.1.

14.13 In Figure 14.3, we have a plot of the results of approximating the velocity of a 1-kg object experiencing a net for

$$F_{\text{net}}^{tv}(t, v_0) = -\alpha v_0,$$

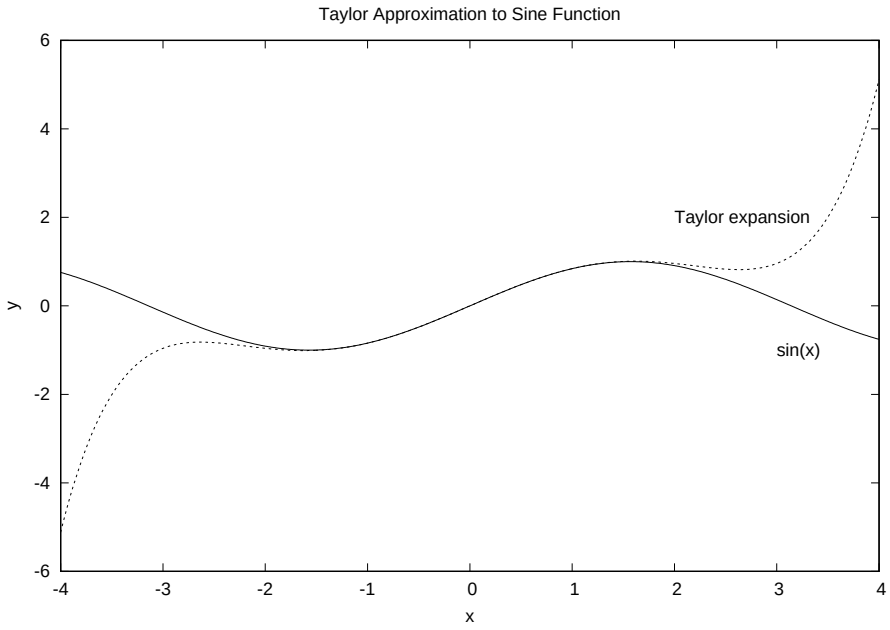


Figure 11.4: Plot of $\sin x$ and first four terms of its Taylor expansion

for $\alpha = 1\,\mathrm{N\ s/m}$, subject to the initial condition $v(0\,\mathrm{s}) = 8\,\mathrm{m/s}$ using the Euler method with $dt = 0.1$ and $dt = 0.5$ compared with the exact solution

$$v(t) = (8\,\mathrm{m/s})e^{-\alpha t}.$$

14.14 In Figure 14.4, we have a plot of the approximating to the velocity function which is the solution of the differential equation

$$\frac{dv(t)}{dt} = \cos\left(t + v(t)\right),$$

subject to the initial condition $v(0) = 0$, with $0 \leq t \leq 3$.

We have made use of the following results from `GHCi` for the value of $v(3)$:

```
ghci> snd (last eulerSolutionDE)
-0.49960475082864086
```

15 Mechanics in One Dimension

15.1 In Figure 15.1, we have a plot of the ball's height as a function of time.

15.2 We have the results in Table 15.1 from executing the following in `GHCi`:

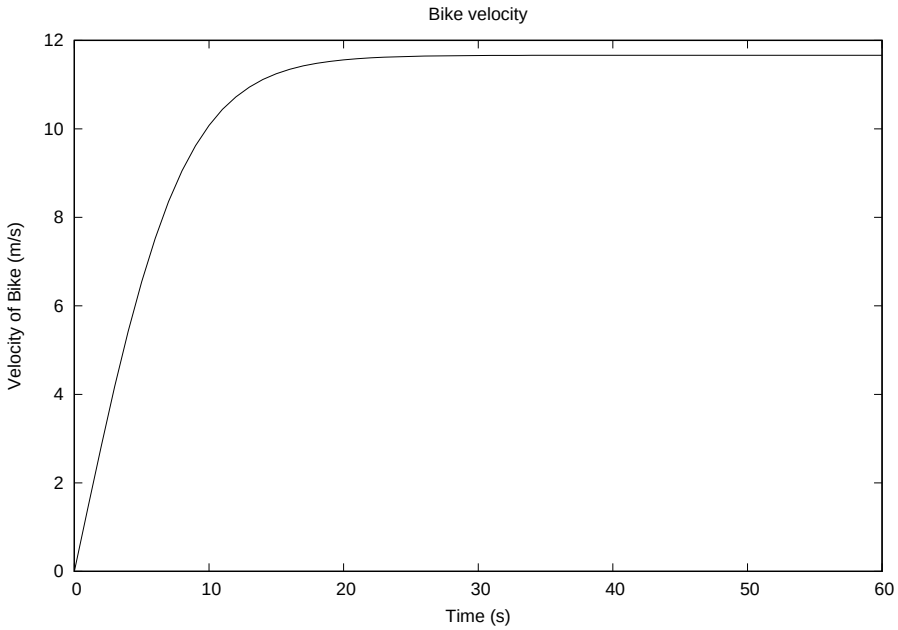


Figure 14.1: Bike velocity as a function of time

t (s)	x(t) (m)	v(t) (m/s)
0.0	0.1000	0.0000
0.1	0.1000	-3.9436
0.2	-0.2944	-7.0006
0.3	-0.9944	3.5348

Table 15.1: Values using Euler method

```
ghci> take 4 pingpongStates1D
```

We can see that, in all cases, the values do match to the first two digits after the decimal point.

15.3 Using the Euler method with

$$\frac{dx(t)}{dt} = v(t)$$
$$\frac{dv(t)}{dt} = -3x(t) + 4\cos 2t - 2v(t),$$

subject to the initial conditions $x(0) = 2$ and $v(0) = 1$, we have the

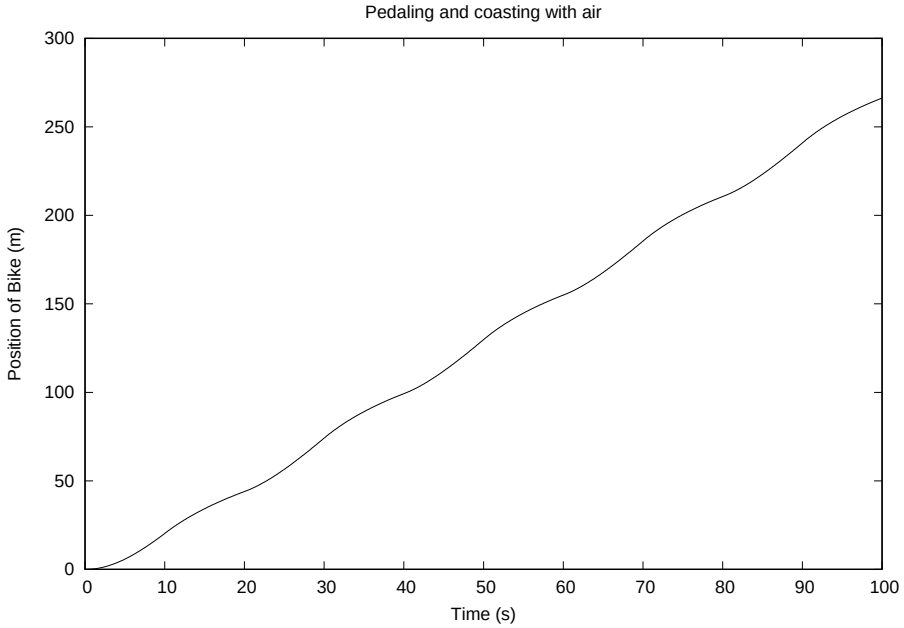


Figure 14.2: Bike position as a function of time

following:

$$
\begin{aligned}
x(0.1) &= x(0) + v(0)\Delta t \\
&= 2.0000 + (1.0000)(0.1) \\
&= 2.1000 \\
v(0.1) &= v(0) + \left(-3x(0) + 4\cos 2t(0) - 2v(0)\right)\Delta t \\
&= 1.0000 + \left((-3)\cdot 2.0000 + 4\cos\left(2\cdot 0\right) - 2\cdot 1.0000\right)(0.1) \\
&= 0.6000 \\
x(0.2) &= x(0.1) + v(0.1)\Delta t \\
&= 2.1000 + (0.6000)(0.1) \\
&= 2.1600 \\
v(0.2) &= v(0.1) + \left(-3x(0.1) + 4\cos 2t(0.1) - 2v(0.1)\right)\Delta t \\
&= 0.6000 + \left((-3)\cdot 2.1000 + 4\cos\left(2\cdot 0.1\right) - 2\cdot 0.6000\right)(0.1) \\
&= 0.2420 \\
x(0.3) &= x(0.2) + v(0.2)\Delta t \\
&= 2.1600 + (0.2420)(0.1) \\
&= 2.1842 \\
v(0.3) &= v(0.2) + \left(-3x(0.2) + 4\cos 2t(0.2) - 2v(0.2)\right)\Delta t \\
&= 0.2420 + \left((-3)\cdot 2.1600 + 4\cos\left(2\cdot 0.2\right) - 2\cdot 0.2420\right)(0.1) \\
&= 0.9940.
\end{aligned}
$$

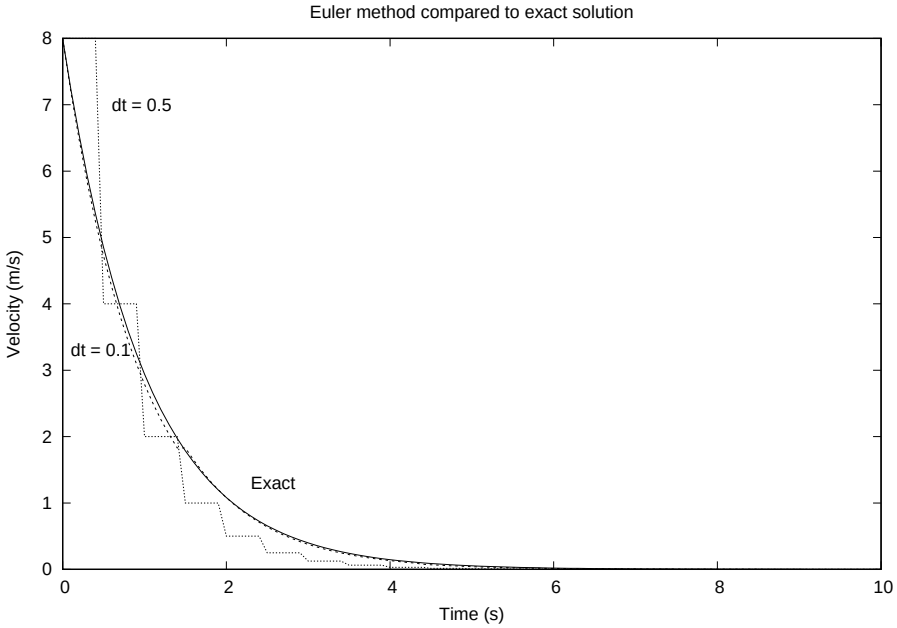


Figure 14.3: Euler method compared with exact solution

The figures are tabulated in Table 15.2.

t (s)	**x(t)**	**v(t)**
0.0	2.0000	1.0000
0.1	2.1000	0.6000
0.2	2.1600	0.2420
0.3	2.1842	-0.0860

Table 15.2: Values using Euler method

15.4 We have the following results from `GHCi`:

```
take 4 (iterate update2 (0.0,2.0000,1.0000))
[(0.0,2.0,1.0),(0.1,2.1,0.6),(0.2,2.16,0.24202663113649653)
↪ ,(0.30000000000000004,2.1842026631136497,-8.595429748964878e-2)]
```

These match closely the values in Table 15.2.

15.5 We must identify the exact solution in order to have something to compare with our numerical solution. Accordingly, we start by identifying the spring force using Hooke's law, so that we have

$$F_{\text{spring}}(x_0) = -kx_0,$$

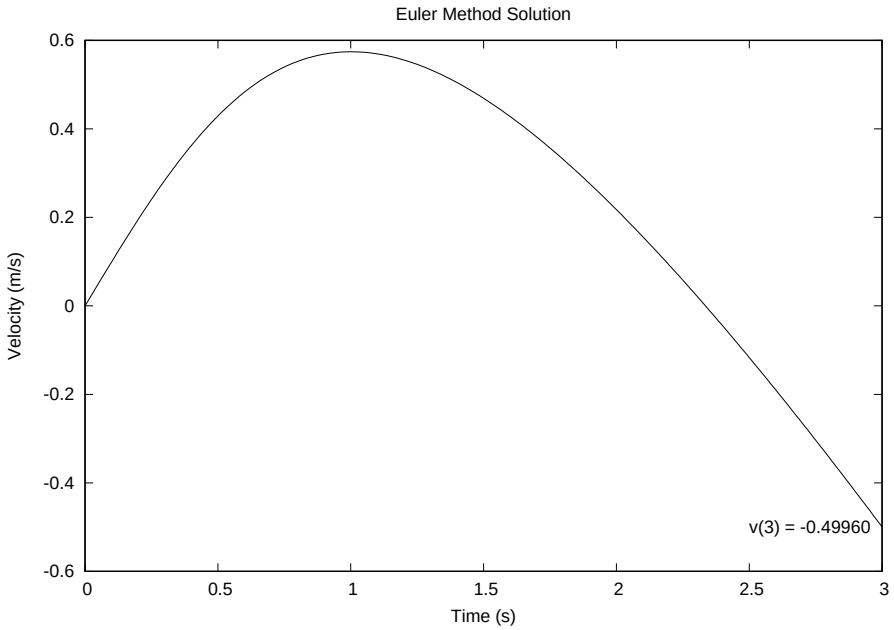


Figure 14.4: Euler method solution to differential equation

as in (15.8), with $k = 100,000\,\mathrm{N/m}$. Since we ignore air resistance and gravity, that is our only force in the differential equation governing the spring's motion. From Newton's second law, we then have

$$F_{\text{spring}}(x) = m\ddot{x},$$

for $m = 3\,\mathrm{kg}$. Our initial conditions are given by the displacement from equilibrium, so that $x(0) = 0.01\,\mathrm{m}$ and the ball is released with no initial velocity, so that $\dot{x(0)} = 0, \mathrm{m/s}$.

Therefore, we have

$$m\ddot{x} = -kx,$$

and since we expect oscillations in the resulting movement, we use the trial solution $x(t) = e^{rt}$, so that $\ddot{x} = r^2e^{rt}$. Thus we have

$$mr^2e^{rt} = -ke^{rt},$$

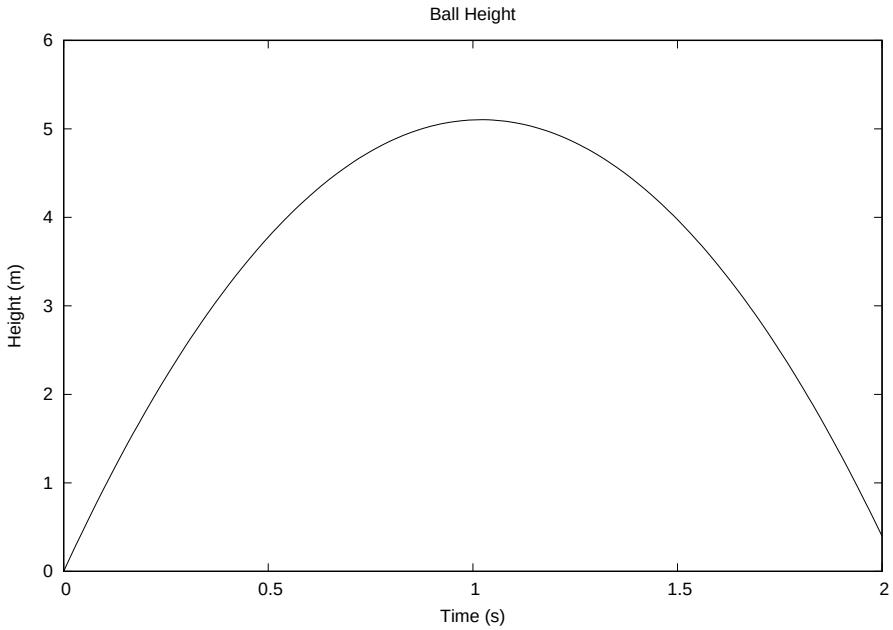


Figure 15.1: Ball height as a function of time

and, since $e^{rt} \neq 0$, we have

$$\begin{aligned} mr^2 + k &= 0 \\ \Leftrightarrow r^2 &= -\frac{k}{m} \\ \Leftrightarrow r &= \pm\sqrt{-\frac{k}{m}} \\ &= \pm i\sqrt{\frac{k}{m}}, \end{aligned}$$

so that our answer becomes, using Euler's formula and the superposition principle,

$$x(t) = a\cos\sqrt{k/m}t + b\sin\sqrt{k/m}t.$$

Now, applying the first initial condition, we have

$$\begin{aligned} x(0) &= a \\ \Rightarrow a &= 0.01, \end{aligned}$$

and from the second, we have

$$\begin{aligned} \dot{x}(0) &= b\sqrt{\frac{k}{m}} \\ \Rightarrow b &= 0. \end{aligned}$$

Therefore, the exact solution is given by

$$\begin{aligned} x(t) &= 0.01 \cos \sqrt{k/m}t \\ &= 0.01 \cos \sqrt{\frac{100,000}{3}}t. \end{aligned}$$

In Figure 15.2, we have a plot of the mass's position as a function of time, with labels for the exact solution, as well as the good approximation (with $\Delta t = 10^{-5}$ s), and bad approximation (with $\Delta t = 5 \times 10^{-3}$ s), using Euler's method.

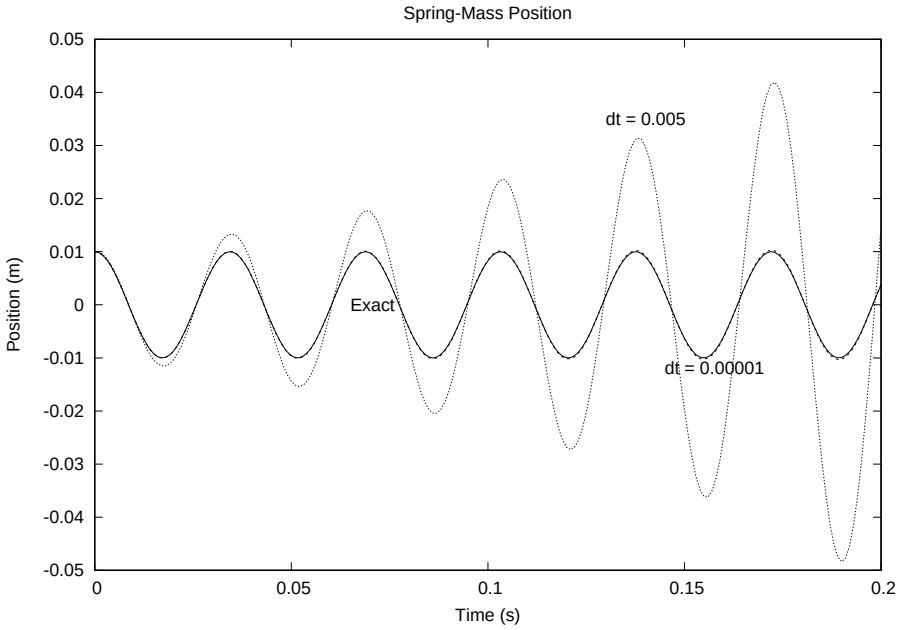


Figure 15.2: Spring-mass position approximations as a function of time